Shrinking the Image Tables (patientsignatures / patinsurances)

Rollout guide for the Admin "Shrink Image Tables" option and the related capture-path changes. Implementation plan and research: `Docs\superpowers\plans\2026-08-04-shrink-ima`

Why this exists

The `.adm` memo files of `patientsignatures` and `patinsurances` grow into the multi-GB range, and measurement showed the growth is **not** waste. A full scan of a production parity snapshot (2026-08-04, walking every in-record 9-byte memo reference) found:

	patientsignatures	patinsurances
records	114,891	259,130
blobs present	114,731 (`image`)	57,450 (`cardimage`)
avg / max blob	65.5 KB / 197 KB	69.6 KB / 485 KB
`.adm` size	7,344 MB	3,907 MB
orphaned + slack	~0.0%	~0.0%
memo block size	8 bytes	8 bytes
used of the 2^{32}-block cap (32 GB at block 8)	22.4%	11.9%

Every sampled blob is a JPEG. The files are ~100% live image data, so packing or re-blocking alone recovers nothing — the size problem is that signatures (monochrome line art) were stored as JPEG **quality 100** (~65 KB each) and insurance cards as raw uncapped scanner output. The shrink is therefore image **re-encoding** (Admin walks every blob, re-encodes, updates in place), followed by `sp_PackTable` to reclaim the replaced memo chains. After the shrink, even the 32 GB cap at block size 8 is only a few percent used, which is why ADS v11 sites need no block-size change at all.

The measurement tool is in the repo: `tools\adm_bloat_scan.ps1` (`powershell -File tools\adm_bloat_scan.ps1 -Tables <path>\patientsignatures.adt,...`) — run it against copies, never against files the server holds exclusively.

Per-customer procedure

Everything below runs from the Admin module logged in as **ADSSYS** (File → Shrink Image Tables). Record the numbers at each step in the findings table at the end of this guide.

1. **Backup.** A verified backup (`sp_BackupDatabase` or file copy) must exist before anything writes. The Restore DB From Backup option (below) is the recovery path.

2. **Deploy the updated Patients module** first, so new signatures/cards/scans are captured small from that moment on (see "Capture-path changes").
3. **Diagnostics** button: record BlockSize, Memo MB and cap % for patientsignatures, patinsurances and every imagesNN table.
4. **Probe SQL default block size** button: record the number — this is the server's real `CREATE TABLE` default (curiosity answer (a) below).
5. **Dry runs** (signatures, then cards): nothing is written; record the projected bytes-before/bytes-after from each summary.
6. **Off-hours: live shrink** both tables. The operation is resumable — it can be cancelled and re-run; already-shrunk blobs fall into "Kept (no gain)" / "Already converted" on the next pass.
7. **All users out**, then **Pack** each table. `sp_PackTable` needs exclusive access — every connection with the table open blocks it, including Admin's own cursors (the option closes its own before packing).
8. **Diagnostics again**: Memo MB should have collapsed to roughly the dry-run "bytes after" figure. Record the after-numbers.
9. Spot-check in Patients: signatures display and print (HIPAA/consent, transcription reports), cards display, zoom and OCR, and the Mp10Web insurance endpoint still serves them.

What the shrink writes: signatures are re-encoded to JPEG quality 60, cards to JPEG quality 75, and a blob is only replaced when the re-encode gains at least 10% — that gate is also what makes repeat runs safe (a second q75→q75 pass gains nothing and writes nothing).

v11 vs v12 expectations

	ADS v11	ADS v12+
Pack call issued	`sp_PackTable( name )`	`sp_PackTable( name, 256 )`
Reclaims replaced memo chains	yes	yes
Memo block size after pack	unchanged (stays 8)	changed to 256 bytes
Blob-count cap after pack	32 GB `.adm` (block 8)	1 TB `.adm` (block 256)
Is that enough?	yes — post-shrink usage is ~3% of the 32 GB cap	yes

Revisit a v11 site only if Diagnostics ever shows the cap % climbing past 50 — the documented escape is upgrading to v12 and packing with the second argument (or the copy-and-swap via `dbCreate()` + `INSERT…SELECT` + `sp_RenameDDObject`, deliberately not built).

New imagesNN tables no longer inherit the problem: `CreateNewImagesTable()` now creates them with 256-byte memo blocks on both versions (SQL `MEMOBLOCKSIZE` on v12+, `dbCreate()` with `_SET_MBLOCKSIZE` on v11).

Capture-path changes (deploy with the shrink)

Signatures are stored as JPEG quality 60 instead of quality 100 (~15–20 KB → a few KB per signature). Repository settings, section `ImageShrink`: `SignatureFormat` (2=JPEG, the shipped default; 13=PNG) and `SignatureQuality` (60). PNG is smaller still for line art and lossless, but switch a site to 13 only after verifying all three PNG consumers there: the encounter grid, the HIPAA/consent print, and a transcription report embedding `Signatures->image`.

Insurance cards are re-encoded to JPEG on save (`CardImageQuality`, default 75), keeping the original whenever the re-encode gains less than 10%. Cards must **stay JPEG** — the Mp10Web endpoints and OCR consume them as JPEG.

Scanned documents headed for an imagesXX table are capped at a 150-DPI-equivalent resolution before storage: longest side 1650 px (`ScanMaxPixels`), JPEG quality 75 (`ScanQuality`). A 600-DPI scanner setting therefore no longer produces multi-MB blobs. PDFs and anything non-JPEG pass through untouched, and existing stored scans are not modified — the cap applies to new scans only.

Restore DB From Backup (Admin, ADSSYS menu) is the safety net for all of this: it runs `sp_RestoreDatabase` against an existing `sp_BackupDatabase` image and restores to a destination directory the operator chooses. Both paths are resolved **by the server** — UNC or server-local drive letters; drives mapped on the workstation are meaningless to it — and the destination should never be the live data directory unless a full rollback is intended.

Per-site findings (fill in as sites are visited)

The Diagnostics screen and probe answer three questions we could not settle from documentation:

- **(a)** What block size does SQL `CREATE TABLE` really default to? (ACE `AdsCreateTable` documents 256; production tables measured 8.) → the Probe button's number.
- **(b)** Did `sp_ModifyTableProperty(…'Table_Memo_Block_Size'…)` ever affect the existing imagesNN files? Expected no (docs say auto-create only) → the imagesNN BlockSize column; 8 (or the probe default) confirms the property is inert for existing files.
- **(c)** On v12, is `CREATE TABLE … MEMOBLOCKSIZE 256` taken literally for ADT (no ×512 folding)? → BlockSize of the first imagesNN created there after this release.

Site / server	ADS ver	(a) probe default	(b) imagesNN block sizes	(c) MEM-OBLOCK-SIZE literal?	Date

Safety notes

- Never run a live shrink without a verified backup.
- Packing requires exclusive access to the table; get every user out first. If a site truly cannot get exclusive access, `sp_PackTableOnline` exists on both v11 and v12 (without the block-size argument), but the offline pack is the default recommendation.
- The shrink itself is re-runnable and cancel-safe by design: the 10% gain gate skips everything already small, so an interrupted run is simply run again.
- The `.adm` file does **not** shrink until the table is packed — a large "orphaned" percentage in `adm_bloat_scan.ps1` between shrink and pack is expected, not a fault.